

Class Objects vs Instances

Why Convert to Class Objects (Ingesting Schema Structures into the SKL):

- **Semantic Enrichment:**
 - **Beyond Tables:** Relational databases (like data warehouses) store data in tables with rows and columns. While powerful for structured querying, they lack explicit semantic meaning. A CUSTOMER_ID column doesn't inherently tell us "this is a person who buys things."
 - **Explicit Meaning:** Converting to OWL/RDF classes and properties in a knowledge graph makes the meaning explicit. ex:Customer rdfs:subClassOf foaf:Person clearly defines a customer. ex:hasPurchased ex:Product clearly defines a relationship.
 - **Context for LLMs:** LLMs thrive on context and relationships. Presenting data as a graph of interconnected entities (class instances) and their properties, rather than isolated rows, provides rich semantic context that significantly improves the LLM's understanding and reasoning.
- **Facilitating RAG (Retrieval Augmented Generation):**
 - **Structured Retrieval:** RAG relies on retrieving relevant information. When data is in a knowledge graph, retrieval can be highly precise using SPARQL queries. The LLM can be trained (or prompted) to generate SPARQL, which is then executed against the graph.
 - **Relationship Mapping:** The core strength of knowledge graphs is **explicit relationships**. Our LLM (LLMs today, SLMs tomorrow), when prompted about "customers who bought product X," can generate a SPARQL query that directly follows the hasPurchased relationship between a Customer instance and a Product instance. This is much harder if the relationship is only implicitly defined by a foreign key join in a relational table.
 - **Explainability:** The retrieved knowledge graph triples (subject-predicate-object) are inherently explainable. We can show the LLM's "thinking" by displaying the specific facts it retrieved from the graph.
- **Leveraging Metaphactory's Strengths:**
 - **Native Format:** Metaphactory is built on knowledge graphs (RDF/OWL). Its tools for visualization, editing, querying (SPARQL), and conversational AI are optimized for this data model. Trying to use it as just a front-end for a relational database would bypass its core capabilities.
 - **Ontology-Driven Development:** Metaphactory encourages defining an ontology (classes, properties, relationships) first, then populating it with data. **This structured approach is what makes complex RAG possible.**
- **Consistency and Interoperability:**
 - As discussed, a common enterprise ontology (expressed as OWL classes and properties) helps unify disparate data sources from our warehouse. **By mapping all warehouse tables/columns to these common classes/properties, we create a single, semantically consistent view of our data.**

Unifying Ontology

Why a Unifying Ontology is Essential for RAG with Disparate KGs:

- **Semantic Interoperability:** Without a common ground, the Rag system (and the LLM it leverages) won't inherently know how concepts in one domain relate to concepts in another. A top-level ontology provides this Rosetta Stone, mapping equivalent or related concepts across different domain-specific KGs.
- **Unified Querying/Reasoning:** When a user asks a question that spans multiple domains, the Rag system needs to know which knowledge graphs (and their respective endpoints) to query and how to combine the results. The unifying ontology guides this process, allowing for federated queries or strategic retrieval from different sources.
- **Preventing Hallucination:** LLMs are prone to hallucinating connections if they don't have explicit, structured knowledge. The unifying ontology acts as a schema for the entire knowledge ecosystem, constraining the LLM's "imagination" to factual and semantically consistent relationships.
- **Improved Entity Linking:** When a user mentions an entity (e.g., "iPhone 15"), the Rag system needs to link it to the correct concept/instance in knowledge graph, regardless of which domain-specific KG it lives in (e.g., "Product Catalog" vs. "Customer Support Issues"). A unifying ontology, with its mappings, helps disambiguate and accurately link these entities.
- **Simplified Prompt Engineering:** Instead of providing the LLM with a fragmented understanding of our knowledge topology, we can point it to a coherent, unified schema. This makes prompt engineering more manageable and robust.
- **Better Explainability:** When the Auto-Mapping provides an answer, the unifying ontology allows us to trace the information flow across domains and sub-domains, making the reasoning behind the answer more transparent.

Recommended Approach:

Note: We are not going to "merge" all of our ontologies into one giant graph. Instead, it's more about **semantic integration and alignment**. Here are the key strategies:

- **The "Core" or "Upper-Level" Ontology:**
 - This is the separate ontology we have implemented (BFO is a standard in OMG and is ISO approved). It's typically a lightweight ontology that defines only the most fundamental, overarching concepts and relationships that apply across *all* our domains.
 - **Purpose:** To act as a central hub, a "roadmap" to our entire knowledge ecosystem. It doesn't contain detailed domain-specific knowledge but rather defines how domains connect.
 - **Examples of Concepts:** Entity, Agent, Event, Location, Time, hasPart, isRelatedTo, occursIn.
 - **Key Properties:** Use owl:sameAs to declare that an entity in Domain A is the

same as an entity in Domain B. Use `rdfs:subClassOf` to show that a class in a domain ontology is a specialization of a class in the core ontology. Use `skos:exactMatch`, `skos:broader`, `skos:narrower` for taxonomic alignment.

- **Explicit Mappings and Alignment:**

- **Within the Core Ontology or a dedicated "Alignment Ontology":** We have defined explicit mappings between concepts and properties in different domain ontologies.
 - Example: If Domain A has a class `Customer` and Domain B has a class `Client`, our core/alignment ontology might state `Customer owl:sameAs Client`.
 - Example: If Domain A has `product:hasPart` and Domain B has `manufacturing:componentOf`, we might align (relate) these properties.
- These mappings are crucial for the Auto-Mapping (specifically, the SPARQL generation tool) to understand how to bridge queries across disparate KGs.

- **Federated Knowledge Graph Setup:**

- Each of our domain KGs having its own IRI endpoint implies they are likely separate SPARQL endpoints or named graphs within a single triple store.
- Our RAG system, when generating SPARQL queries, will need to know which endpoints to query. The unifying ontology helps here:
 - If a query involves a concept defined in the core ontology, the Auto-Mapping might start by querying all relevant endpoints, or use smarter routing based on the query.
 - For Metaphactory specifically, we have defined multiple knowledge graph connections (to each IRI endpoint). The conversation agent then, guided by the unified ontology, determines which connections to leverage for a given query.

Our Approach:

We originally built using a core domain-enterprise ontology ([Ontology | Core | Domain](#)) from which all business-specific domain ontologies are built and their sub-domain ontologies will inherit. This approach was built in from the beginning and it carries all the core taxonomies and their core vocabularies as part of the inheritance. This is our solution for implementing a Unifying Ontology for RAG navigation.

This setup addresses many of the key principles:

- **Existing Hierarchical Structure:** The inheritance (`rdfs:subClassOf`, `rdfs:subPropertyOf`) already provides the vertical navigation pathways from general enterprise concepts down to specific domain and sub-domain concepts.
- **Built-in Interoperability:** By requiring all domain-specific ontologies to inherit from or align with this core domain-enterprise ontology, we inherently enforced a level of semantic consistency and interoperability across our different knowledge domains. This forms our "horizontal" navigation basis.
- **Common Vocabulary:** The core domain-enterprise ontology provides a shared vocabulary for concepts that span our entire organization (e.g., Product, Party, Persona, Customer, Location etc). These core structures form the common ground for our RAG system.

Leveraging the Unifying Ontology for RAG:

- **Designate it as the Primary Context Ontology:** In Metaphactory this domain-enterprise ontology should be the primary `contextOntology` or schema reference given to our Gemini-powered RAG agent. This tells Gemini: "Here is the master blueprint of our entire knowledge base."
- **Ensure All Domain KGs Are Accessible:** Make sure that the data instantiated from all our domain-specific and sub-domain ontologies (which inherit from our enterprise ontology) are accessible to our RAG system. This could be:
 - All in one large triple store (perhaps using named graphs for different domains/sub-domains).
 - Federated across multiple SPARQL endpoints, where each endpoint hosts a domain-specific KG.
- **Configure RAG Tools to Utilize Inheritance and Mappings:**
 - **SPARQL Translation Tool:** This tool needs to be aware of the inheritance hierarchy. When Gemini (via this tool) processes a query, it should be able to:
 - **Traverse Down:** If a user asks about `EnterpriseProduct`, it can understand that `ElectronicsProduct` (from a sub-domain) is a type of `EnterpriseProduct` and generate SPARQL that queries for `?x rdf:type electronics:ElectronicsProduct` or `?x rdf:type sales:SpecificProduct` as needed. This often involves using

SPARQL property paths like (rdfs:subClassOf)*.

- **Recognize Cross-Domain Connections:** If our core enterprise ontology defines relationships or owl:sameAs axioms between concepts that appear in different inheriting domain ontologies, the SPARQL translation should be able to leverage these to formulate federated or cross-graph queries.
- **Entity Linking Tool:** This tool will use the entire unified vocabulary (from the enterprise ontology down to the sub-domains) to accurately link natural language terms to specific IRIs in our knowledge graph.
- **Prompt Engineering:**
 - Our main system prompt for Gemini should emphasize that it has access to a comprehensive knowledge graph structured around a core enterprise ontology, with hierarchical inheritance to specific domains and sub-domains.
 - Example: The prompt could evolve into an expert AI assistant with access to our enterprise knowledge graph. This graph is organized around a core ontology that defines our key business concepts. All our specialized domain and sub-domain knowledge (e.g., about products, resources, assets, customers, operational processes, services, value chains, and supply chains) inherits directly from this core, allowing a business SME to seamlessly navigate from high-level enterprise views down to specific details, and across related business functions.

Potential Considerations/Refinements:

- **Consistency of Inheritance:** Double-check that the inheritance hierarchies are logically sound and consistent across all our domain and sub-domain ontologies. Any inconsistencies here will confuse the RAG system.
- **Granularity of Mappings:** Ensure that the core domain-enterprise ontology has sufficient mappings (owl:sameAs, skos:exactMatch, etc.) if specific instances or concepts are referred to differently but mean the same thing across different inheriting domain KGs. While inheritance helps with classes, we might still need explicit instance-level mappings.
- **Documentation:** Clear documentation of our core domain-enterprise ontology and how it relates to the specialized domain ontologies will be invaluable for configuring and debugging our RAG system.

Integrating VZ Gemini into Metaphactory

Benefits of Integrating Metaphactory with GenAI (VZ Gemini in our case):

Knowledge Graph Foundation:

Metaphactory is a knowledge graph platform. Knowledge graphs provide structured, contextualized data. LLMs perform much better when they have access to accurate, well-organized external knowledge. This is the core of Retrieval Augmented Generation (RAG), where the knowledge graph serves as the "retrieval" mechanism to ground the LLM's responses and prevent hallucination.

Conversational AI Features:

Metaphactory has built-in capabilities for conversational AI. Version 5.7, in particular, introduced a more flexible mechanism for configuring multiple language model instances and conversation agents. This means we can plug in different LLMs (like Gemini, OpenAI, etc.) and configure how they interact with our knowledge graph.

Extensibility and APIs:

Metaphactory is designed for extensibility. It exposes REST APIs for querying and manipulating RDF data (our knowledge graph), and allows for custom web components and backend extensions. This provides the necessary hooks to connect to external services like the Gemini API.

Semantic Modeling:

We have designed a component model for capturing business oriented knowledge. We embedded all of the domain-specific ontologies with a unifying ontology (see prior tab) to support RAG-based learning and traversal for semantic search. Key benefits of the Semantic Knowledge Layer include:

- Improve entity linking in natural language queries.
- Help the LLM understand the context and relationships in our data.
- Translate natural language questions into SPARQL queries that the knowledge graph can execute.
- Enhance AI Governance by combining LLMs with a knowledge graph we can improve control over the data and how it's organized in vector DBs used by LLMs and Agents while extending explainable AI insights by tracing back the LLM's responses to facts in our semantic knowledge layer (federated graphs).

Integrating VZ Gemini into Metaphactory v5.7

The general approach involves leveraging Metaphactory's "AI Services" and its extensibility points:

- Configure Gemini as a Language Model Service:
 - Metaphactory's "AI Services" administration page allows us to configure external language models.
 - Create a new service instance for Gemini, specifying its API endpoint and authentication mechanism (likely an API key).
 - This configuration tells Metaphactory how to send requests to and receive responses from the VZ Gemini API.
- Set up a Conversation Agent (if applicable):
 - If we want to enable a conversational interface (like a chatbot) within Metaphactory, we need to configure a conversation agent.
 - This agent will utilize the Gemini language model service we have configured.
 - Crucially, we will need to specify the IRI (Internationalized Resource Identifier) of the **unifying ontology** ([Ontology | Core | Domain](#)) within Metaphactory that the conversation agent should use as the initial entry point for exploring and searching for relevant context, entity linking, and SPARQL translation. This is where our semantic knowledge layer (federated knowledge graph(s)) becomes the backbone for VZ Gemini's understanding.
- Leverage Metaphactory's Tools for RAG:
 - Metaphactory's conversational AI often uses "tools" to interact with the knowledge graph. These tools can include:
 - 🌕 **Link Entities Tool:** To identify and link named entities from a user's natural language query to specific entities in our knowledge graph.
 - 🌕 **SPARQL Translation Tool:** To translate natural language questions into SPARQL queries that can retrieve answers directly from our knowledge graph.
 - 🌕 **SPARQL Query Execution Tool:** To execute the generated SPARQL queries.
 - 🌕 **Describe Entity Tool:** To retrieve descriptions or properties of entities from the knowledge graph.

- Configure the conversation agent to use these tools. When a user asks a question, the agent (powered by VZ Gemini) will use these tools to query the knowledge graph for “relevant” concepts before generating a response.
- Customization and Refinement:
 - **Ontology Enhancements:** Ongoing refinement and elaboration of domain specific ontologies to provide better context and improve entity linking and SPARQL translation.
 - **Entity Descriptions:** Ensure our knowledge graph has rich and accurate descriptions for entities, which Gemini can leverage for more informative responses.
 - **Entity Detection Examples:** Adapt entity detection examples to our specific domain within Metaphactory to enhance how the system identifies and links entities.
- Extend Metaphactory UI Components:
 - Metaphactory provides a UI component (<mp-conversational-ai>) for building chat interfaces. We may want to customize this component to display the interaction between VZ Gemini and the semantic knowledge layer to see how it's navigating and learning (see UX for AI Training tab)

RAG Tooling for Navigation:

- **SPARQL Generation Tool:** This tool (which Gemini helps to invoke) is taught how to construct SPARQL queries that leverage the owl:sameAs links, rdfs:subClassOf hierarchies, and potentially SERVICE clauses (for federated queries across distinct endpoints) or GRAPH clauses (for named graphs). The prompt for this tool is critical for conveying the overall schema.
- **Entity Linking Tool:** This tool uses the combined vocabulary from our core and domain ontologies to accurately identify and link terms in the user's natural language query to specific IRIs in our knowledge graph.
- **Hierarchical Traversal Tools:** Specialized tools can be built to explicitly navigate rdfs:subClassOf, skos:broader/skos:narrower relationships, helping Gemini refine queries or provide more granular information.
- **Note 1:** The SPARQL Translation Tool needs to be intelligent enough to leverage the rdfs:subClassOf, owl:sameAs, and other axiomatic relationships defined across our core and domain-specific (business) ontologies by leveraging the unifying ontology.
- **Note 2:** The Entity Linking Tool will need to draw from the combined vocabulary of all these interconnected ontologies.

UX for AI Training

UX Design for AI Training (Auto-Mapping):

This document presents a user experience (UX) design for an AI-powered ontology mapping tool. The goal is to create an intuitive and efficient interface that allows users to guide a large language model (LLM), such as Google Gemini, in establishing relationships between top-down enterprise ontologies and bottom-up data structure models (RDFs).

This UX design provides a robust framework for integrating Google Gemini into Metaphactory to create a powerful and user-friendly auto-mapping tool. The focus on user guidance and explainable AI will be key to building trust in the system and achieving high-quality ontology mappings at scale.

This design is intended to be shared with Metaphacts as they begin preparing an early release of their latest product Metaphactory v5.8. The goal is to leverage existing UI components and design an experience which is efficient, simple to use, cohesive, comprehensive, and collaborative.

Core UX Principles

The design is guided by the following principles:

- **Efficiency and Speed:** The interface should empower power users to perform mapping tasks quickly and with minimal friction.
- **Clarity and Intuitiveness:** The interface should be easy to understand for users who may not be ontology experts but are subject matter experts in their respective domains.
- **Guided AI Training:** The UX should provide mechanisms for users to "teach" the AI, not just accept or reject its suggestions. This includes providing examples, setting constraints, and understanding the AI's reasoning. The AI needs to incorporate negative and positive feedback. A key concern is teaching the AI to traverse the topology of Top-Down ontologies by leveraging our Unifying Ontology (Domain-Specific Ontology).
- **Collaborative Workflow:** The design should support a collaborative workflow where different user roles (e.g., ontology engineers, data stewards) can contribute to the mapping process.

User Personas and Scenarios

- **Priya, the Ontology Engineer:** Priya is responsible for the overall health and consistency of the enterprise ontologies. She needs to train the AI with high-quality mappings, review its suggestions, and establish the foundational relationships that others will build upon.
- **David, the Data Steward:** David is a subject matter expert for a specific business domain (e.g., "Customer Billing"). He is responsible for ensuring the data in his domain is accurately represented and mapped. He will primarily be validating and correcting the AI's suggestions within his area of expertise.
- **Maria, the Business Analyst:** Maria needs to understand the relationships between different data assets to generate reports and insights. She will be a consumer of the mapped ontologies, so the interface should provide clear visualizations of the relationships.
- **Note:** These 3 roles will be performed by the Knowledge Engineering team for MVP1

Information Architecture and Workflow

The proposed UX introduces a new "Auto-Mapping" section within the Metaphactory navigation.

1. **Dashboard:** The landing page for the Auto-Mapping section. It provides an at-a-glance view of the mapping progress, including:
 - Overall mapping completeness.
 - Number of pending suggestions for review.
 - Activity feed of recent mappings.
 - Links to start a new mapping task or continue a previous one.
2. **Mapping Workspace:** The core of the UX. This is where the mapping and AI training takes place. It's a two-panel layout:
 - **Left Panel:** A tree-view of the Top-Down Ontology.
 - **Right Panel:** A tree-view of the Bottom-Up Ontology (Projects, Databases, Tables, Columns).
3. **AI Assistant Panel:** A collapsible panel on the right side of the Mapping Workspace that presents the AI's suggestions, explanations, and confidence scores.
4. **Review and Validation Workflow:**
 - When an ontology engineer or data steward creates a mapping, they can assign it to another user for review.
 - Users receive notifications for pending reviews.
 - A dedicated "Review" queue allows users to see all mappings awaiting their approval.

Wireframes and a step by step walk through

The Dashboard

The user starts at the Auto-Mapping Dashboard.

From here, the user can get a quick overview of the mapping status and can start a new mapping task. Let's assume the user clicks on "New Mapping".

The Mapping Workspace

The user is taken to the Mapping Workspace. The left panel shows the Top-Down ontology, and the right panel shows the Bottom-Up ontology.

Creating a Mapping

The user can create a mapping by dragging and dropping a node from the left panel to the right panel. For example, Priya drags the "Customer" class from the Top-Down ontology and drops it onto the "customers" table in the Bottom-Up ontology.

A modal window appears, allowing the user to specify the relationship type (e.g., "hasDataIn", "isModeledBy"). This action serves as a "lesson" for the AI.

AI-Powered Suggestions

As the user creates mappings, the AI learns and starts to provide suggestions. The AI Assistant panel on the right will show these suggestions, along with a **confidence score** and an **explanation of its reasoning**.

User Actions:

- **Accept:** Accepts the suggestion and creates the mapping.
- **Reject:** Rejects the suggestion and provides a reason, which further trains the AI.
- **Edit:** Accepts the suggestion but modifies the relationship type or other parameters.

Review and Collaboration

Once a set of mappings is complete, the user can request a review from a colleague. The reviewer will see the mappings in a "pending review" state and can approve or reject them, providing comments as needed.

Interaction Model and AI Guidance

- **Teaching by Example:** Every manual mapping action is a learning opportunity for the AI.
- **Constraints and Rules:** Users can define constraints to guide the AI. For example, "The 'Financial' class in the top-down ontology should only be mapped to tables in the 'finance_db' project."
- **Explainable AI:** The AI will provide a brief explanation for each suggestion, such as "I suggest this mapping because the 'customer' table contains columns named 'first_name', 'last_name', and 'email_address', which are properties of the 'Customer' class."
- **Confidence Scores:** The AI will present its suggestions with a confidence score (e.g., 95% confident), allowing users to prioritize their review efforts.

UX Mockup (v1.0)

Top-Down Ontology (Domain & Sub-Domains)

- **Sales.Party.Customer** (Concept)
- **Sales.Party.Customer.Contact.Email** (Sub-Concept)
- **Sales.Party.Customer.Contact.Phone** (Sub-Concept)
- **Sales.Party.Customer.Demographics.RegistrationDate** (Sub-Concept)
- **Sales.Party.Customer.Location.AddressLine1** (Sub-Concept)
- **Sales.Party.Customer.Location.City** (Sub-Concept)
- **Sales.Product.SalesItem** (Concept)
- **Product.Attributes.Description** (Sub-Concept)
- **Product.Inventory.StockQuantity** (Sub-Concept)
- **Product.Pricing.CurrentPrice** (Sub-Concept)
- **Sales.Process.SalesOrder** (Concept)
- **OrderManagement.Order.Details.Quantity** (Sub-Concept)
- **OrderManagement.Order.Details.ProductSKU** (Sub-Concept)
- **OrderManagement.Order.Financial.Amount** (Sub-Concept)
- **OrderManagement.Order.Payment.Method** (Sub-Concept)
- **Sales.Event.SalesTransaction** (Concept)
- **Sales.Metric.SalesRevenue** (Metric)
- **Sales.Metric.AverageOrderValue** (Metric)
- **Sales.Persona.HighValueCustomer** (Persona)
- **Sales.Persona.NewCustomerSegment** (Persona)
- **Marketing.Party.Prospect** (Concept)
- **Marketing.Party.CustomerSegment** (Concept)

- **Marketing.Process.Campaign** (Concept)
- **Marketing.Event.CustomerInteraction** (Concept)
- **Legal.Asset.Contract** (Concept)
- **Legal.Issue.ComplianceViolation** (Concept)
- **Finance.Asset.Invoice** (Concept)
- **Finance.Process.Payment** (Concept)
- **Finance.Metric.Profit** (Metric)
- **Finance.Metric.Cost** (Metric)
- **OrderManagement.Process.OrderCreation** (Concept)
- **OrderManagement.Process.OrderLifecycle** (Concept)
- **OrderManagement.Product.OrderedItem** (Concept)
- **OrderManagement.Party.OrderCustomer** (Concept)
- **OrderFulfillment.Process.Picking** (Concept)
- **OrderFulfillment.Process.Packing** (Concept)
- **OrderFulfillment.Process.Shipping** (Concept)
- **OrderFulfillment.Asset.Warehouse** (Concept)
- **Logistics.Process.Delivery** (Concept)
- **Logistics.Location.DistributionCenter** (Concept)
- **Logistics.Asset.Vehicle** (Concept)
- **Logistics.Event.ShipmentTracking** (Concept)
- **Logistics.Location.City** (Sub-Concept)
- **Logistics.Location.Address** (Sub-Concept)
- **Distribution.Process.InventoryTransfer** (Concept)
- **Distribution.Location.RetailStore** (Concept)
- **IT.Asset.System** (Concept)
- **IT.Service.SupportTicket** (Concept)
- **IT.Party.ITUser** (Concept)
- **IT.Event.SystemLog** (Concept)
- **IT.Service.Ticket.Status** (Sub-Concept)
- **IT.Service.Ticket.Priority** (Sub-Concept)
- **HR.Party.Employee** (Concept)
- **HR.Party.Manager** (Concept)
- **HR.Process.Hiring** (Concept)
- **HR.Event.PerformanceReview** (Concept)
- **HR.Asset.EmployeeRecord** (Concept)
- **HR.Metric.EmployeeCount** (Metric)
- **HR.Location.OfficeLocation** (Concept)
- **HR.Party.Employee.JobTitle** (Sub-Concept)
- **HR.Party.Employee.Department** (Sub-Concept)
- **HR.Party.Employee.HireDate** (Sub-Concept)
- **HR.Party.Employee.Salary** (Sub-Concept)
- **HR.Party.Employee.ManagerID** (Sub-Concept)
- **Location.Physical.City** (Concept)
- **Geographic Hierarchy** (Hierarchy)
- **Service** (Concept)

Bottom-Up Ontology (Schema & Instances)

- **users_table.user_id** (Schema Element)
- **users_table.customer_name** (Schema Element)
- **users_table.email_address** (Schema Element)
- **users_table.registration_date** (Schema Element)
- **users_table.last_login_date** (Schema Element)
- **users_table.customer_segment_id** (Schema Element)
- **users_table.address_line1** (Schema Element)
- **users_table.city** (Schema Element)
- **users_table.country_code** (Schema Element)
- **users_table.phone_number** (Schema Element)
- **products_catalog.product_sku** (Schema Element)
- **products_catalog.item_description** (Schema Element)
- **products_catalog.category** (Schema Element)
- **products_catalog.price** (Schema Element)
- **products_catalog.stock_quantity** (Schema Element)
- **products_catalog.supplier_id** (Schema Element)
- **products_catalog.weight_kg** (Schema Element)
- **products_catalog.release_date** (Schema Element)
- **products_catalog.is_active** (Schema Element)
- **products_catalog.product_image_url** (Schema Element)
- **sales_transactions.order_id** (Schema Element)
- **sales_transactions.transaction_date** (Schema Element)
- **sales_transactions.customer_id** (Schema Element)
- **sales_transactions.product_sku** (Schema Element)
- **sales_transactions.quantity** (Schema Element)
- **sales_transactions.amount** (Schema Element)
- **sales_transactions.payment_method** (Schema Element)
- **sales_transactions.shipping_address_id** (Schema Element)
- **sales_transactions.status** (Schema Element)
- **sales_transactions.discount_applied** (Schema Element)
- **support_tickets.ticket_id** (Schema Element)
- **support_tickets.issue_type** (Schema Element)
- **support_tickets.description** (Schema Element)
- **support_tickets.raised_by_user_id** (Schema Element)
- **support_tickets.assigned_to_employee_id** (Schema Element)
- **support_tickets.status** (Schema Element)
- **support_tickets.priority** (Schema Element)
- **support_tickets.resolution_date** (Schema Element)
- **support_tickets.channel** (Schema Element)
- **support_tickets.last_updated_by** (Schema Element)
- **geo_data.country** (Schema Element)
- **geo_data.state_province** (Schema Element)
- **geo_data.city** (Schema Element)
- **geo_data.zip_code** (Schema Element)
- **geo_data.latitude** (Schema Element)
- **geo_data.longitude** (Schema Element)
- **geo_data.region_name** (Schema Element)
- **geo_data.population** (Schema Element)
- **geo_data.timezone** (Schema Element)
- **geo_data.continent** (Schema Element)
- **hr_system.employee_id** (Schema Element)

- **hr_system.employee_name** (Schema Element)
- **hr_system.department** (Schema Element)
- **hr_system.job_title** (Schema Element)
- **hr_system.hire_date** (Schema Element)
- **hr_system.salary** (Schema Element)
- **hr_system.manager_id** (Schema Element)
- **hr_system.email** (Schema Element)
- **hr_system.status** (Schema Element)
- **hr_system.office_location** (Schema Element)
- **SUM(sales_transactions.amount)** (Metric)
- **AVG(sales_transactions.amount)** (Metric)
- **geo_data.country -> geo_data.state_province -> geo_data.city** (Hierarchy)
- **Customer_Type = "High Value"** (Persona)
- **users_table.registration_date > [last 30 days]** (Persona)

🕒 AI Mapping Progress



Progress: 0% | Confidence: 0%

Ready to start mapping.

Start Automated Mapping

Progress: 100% | Confidence: 95%

Mapping complete! Review identified relationships and their polymorphic meanings.

100%

Auto Mapping Dashboard

Identified Relationships (Grouped by Schema)

Schema: AVG(sales_transactions)

Top-Down Concept (Node)	Bottom-Up Element	Mapping Type	Context	Confidence
Sales.Metric.AverageOrderValue	AVG(sales_transactions.amount)	is_calculated_as	Financial Reporting	High

Schema: Instances

Top-Down Concept (Node)	Bottom-Up Element	Mapping Type	Context	Confidence
Sales.Persona.HighValue Customer	Customer_Type = "High Value"	is_defined_by_rule	Customer Segmentation	High

Schema: SUM(sales_transactions)

Top-Down Concept (Node)	Bottom-Up Element	Mapping Type	Context	Confidence
Sales.Metric.SalesRevenue	SUM(sales_transactions.amount)	is_calculated_as	Financial Reporting	High

Schema: geo_data

Top-Down Concept (Node)	Bottom-Up Element	Mapping Type	Context	Confidence
Location.Physical.City	geo_data.city	is_represented_by	General Geographic Data	High
Logistics.Location.City	geo_data.city	is_represented_by	Geographic Routing	High
Logistics.Location.City	geo_data.city	is_represented_by	Geographic Routing	High

Schema: hr_system

Top-Down Concept (Node)	Bottom-Up Element	Mapping Type	Context	Confidence
HR.Party.Employee.Department	hr_system.department	is_represented_by	Organizational Structure	High
HR.Party.Employee	hr_system.email	has_contact_info	Employee Communication	High

HR.Party.Employee	hr_system.employee_id	is_identified_by	Employee Record	High
HR.Party.Employee	hr_system.employee_id	is_identified_by	HR Record	High
IT.Party.ITUser	hr_system.employee_id	has_system_access	IT User Management	High
IT.Party.ITUser	hr_system.employee_id	has_system_access	IT User Management	High
HR.Party.Employee	hr_system.employee_name	has_attribute	Employee Profile	High
HR.Party.Employee.HireDate	hr_system.hire_date	is_represented_by	Employment Lifecycle	High
HR.Party.Employee.JobTitle	hr_system.job_title	is_represented_by	Role Definition	High
HR.Party.Employee.ManagerID	hr_system.manager_id	reports_to	Reporting Structure	High
HR.Location.OfficeLocation	hr_system.office_location	is_represented_by	Workplace	High
HR.Party.Employee.Salary	hr_system.salary	is_represented_by	Compensation	High

Schema: products_catalog

Top-Down Concept (Node)	Bottom-Up Element	Mapping Type	Context	Confidence
Finance.Metric.Cost	products_catalog.price	influences	Cost Analysis	Low
Sales.Product.SalesItem	products_catalog.product_sku	is_identified_by	Product Sales	High
Distribution.Process.InventoryTransfer	products_catalog.stock_quantity	affects_quantity_of	Stock Movement	Medium
OrderFulfillment.Asset.Warehouse	products_catalog.stock_quantity	manages_inventory_at	Inventory Location	High

Schema: sales_transactions

Top-Down Concept (Node)	Bottom-Up Element	Mapping Type	Context	Confidence
Finance.Metric.Profit	sales_transactions.amount	contributes_to	Profit Calculation	Medium
Finance.Metric.Profit	sales_transactions.amount	influences_calculation	Profit Analysis	Medium

OrderManagement.Order.Financial.Amount	sales_transactions.amount	is_represented_by	Order Financials	High
OrderManagement.Order.Financial.Amount	sales_transactions.amount	is_represented_by	Order Transaction Value	High
Sales.Metric.SalesRevenue	sales_transactions.amount	contributes_to_metric	Sales Reporting	High
Legal.Asset.Contract	sales_transactions.order_id	is_referenced_by	Legal Compliance	Low
Sales.Process.SalesOrder	sales_transactions.order_id	is_represented_by	Order Tracking	High
Finance.Process.Payment	sales_transactions.payment_method	uses_method	Payment Processing	High
OrderManagement.Order.Details.ProductSKU	sales_transactions.product_sku	is_represented_by	Order Item Details	High
OrderManagement.Order.Details.Quantity	sales_transactions.quantity	is_represented_by	Order Item Details	High
Logistics.Process.Delivery	sales_transactions.shipping_address_id	targets_delivery_to	Delivery Operations	High
OrderFulfillment.Process.Shipping	sales_transactions.shipping_address_id	targets_address	Shipping Logistics	High
Logistics.Event.ShipmentTracking	sales_transactions.status	updates_status_via	Shipment Status	Medium
OrderManagement.Process.Order Lifecycle	sales_transactions.status	has_stage	Order Workflow	High
OrderManagement.Process.Order Creation	sales_transactions.transaction_date	initiated_on	Order Lifecycle	High
Sales.Event.SalesTransaction	sales_transactions.transaction_date	occurred_on	Transaction Reporting	High

Schema: support_tickets

Top-Down Concept (Node)	Bottom-Up Element	Mapping Type	Context	Confidence
Service	support_tickets.assigned_to_employee_id	assigned_to_employee	Service Workflow	High
IT.Service.SupportTicket	support_tickets.issue_type	has_type	Issue Categorization	High

IT.Service.Ticket.Priority	support_tickets.priority	is_represented_by	Ticket Workflow	High
IT.Service.Ticket.Status	support_tickets.status	is_represented_by	Ticket Workflow	High
IT.Service.SupportTicket	support_tickets.ticket_id	is_tracked_by	Helpdesk Operations	High

Schema: users_table

Top-Down Concept (Node)	Bottom-Up Element	Mapping Type	Context	Confidence
Logistics.Location.Address	users_table.address_line1	is_represented_by	Delivery Address	High
Sales.Party.Customer.Location.AddressLine1	users_table.address_line1	is_represented_by	Customer Address	High
Sales.Party.Customer.Location.City	users_table.city	is_represented_by	Customer Address	High
Sales.Party.Customer.Location.City	users_table.city	is_represented_by	Customer Location	High
Sales.Party.Customer	users_table.customer_name	has_attribute	Customer Profile	High
Marketing.Party.CustomerSegment	users_table.customer_segment_id	is_defined_by	Segmentation Strategy	High
Marketing.Party.Prospect	users_table.email_address	is_target_for	Campaign Management	Medium
Sales.Party.Customer.Contact.Email	users_table.email_address	is_represented_by	Customer Communication	High
Marketing.Event.CustomerInteraction	users_table.last_login_date	is_indicated_by	Engagement Tracking	Medium
Sales.Party.Customer.Contact.Phone	users_table.phone_number	is_represented_by	Customer Communication	High
Sales.Party.Customer.Demographics.RegistrationDate	users_table.registration_date	is_represented_by	Customer Lifecycle	High
Sales.Persona.NewCustomerSegment	users_table.registration_date > [last 30 days]	is_defined_by_rule	Customer Segmentation	High
Marketing.Party.Prospect	users_table.user_id	is_potential_target	Marketing Prospect ID	Medium
Sales.Party.Customer	users_table.user_id	is_represented_by	Customer Identification	High

Sales.Party.Customer

users_table.user_id

UX for Semantic Search

Target State Design

UI Blueprint for Metaphactory Semantic Search

The implementation will be based on creating a custom Metaphactory dashboard that integrates standard components, custom widgets, and powerful SPARQL queries to deliver the desired user experience.

1. The Home Screen - Starting the Search Journey

- **Main Search Bar:** This will be implemented using Metaphactory's default search component. The underlying query engine will be configured to use a federated SPARQL endpoint, which understands the ontology's semantics. The placeholder text will be a configurable property.
- **Conversational Guidance:** A custom-developed UI widget (a simple HTML/CSS component) will listen for user input in the search bar. Based on the query string length or a specific keypress, it will dynamically display the natural language guidance text, "I can find documents, data sets, presentations, and more based on your intent."
- **"Your Knowledge Feed":** This is a custom dashboard section.
 - **Personalization:** The dashboard will be configured to identify the logged-in user's persona (e.g., "Marketing Analyst") from their user profile or a shared knowledge artifact.
 - **Dynamic Cards:** The cards for "Trending in Your Domain" and "Popular Searches" will be powered by a SPARQL query that pulls pre-defined knowledge:SearchArtifacts from the graph. The query will use filters based on the user's business domain (?searchArtifact a knowledge:SearchArtifact . ?searchArtifact knowledge:usedByDepartment "Marketing"), and will sort results by a knowledge:usageCount property to show trending items. The Run Search button on each card will execute the saved query directly, populating the results page.

2. The Search Results Page - Dynamic Filtering and Contextual Relevance

- **Header and Natural Language Feedback:** This will be a custom UI component at the top of the results page.
 - It will take the user's initial natural language query and dynamically generate a summary string. For example, it will use SPARQL to count the number of returned assets (SELECT (COUNT(?asset) AS ?count) ...) and identify the user's persona to display a sentence like, "I found {{count}} assets related to '{{originalQuery}}', and have prioritized those most relevant to your {{userPersona}} persona."
- **Main Results Panel:** This will be a configurable Metaphactory search results view.
 - **Card-based Layout:** A custom display template will be used to render each search result as a clean card. The card's content (Title, Snippet, Metadata,

Source, Author) will be populated by data from the knowledge graph using SPARQL.

- **Trust Score:** The trust score or data quality icon will be rendered based on a property associated with the data source or the knowledge artifact itself, pulled from the knowledge graph (e.g., `?asset a schema:DigitalDocument . ?asset knowledge:hasDataQuality "Gold"`).
- **Dynamic Filters Sidebar:** This is where Metaphactory's facet engine shines.
 - **Persona & Content Relevance Sliders:** This requires a custom faceted search component. The slider would operate on numerical properties like `knowledge:relevanceScore` or `knowledge:projectAffinity`. A developer would configure the slider's range and the SPARQL filter it applies to the main search query, dynamically updating the results. The natural language feedback would be a custom tooltip, which is a standard UI element.
 - **Other Filters:** Standard Metaphactory faceted search components can be used for Source, Date Range, and Topic. These facets are automatically generated from the properties of the data in the knowledge graph, providing an intuitive, data-driven filtering experience.

3. The Knowledge Graph Navigator - Visual Discovery

- **Integration:** This will be a standard Metaphactory graph visualization component.
- **Functionality:** The component will be configured to display the relationships between the results and key concepts (People, Projects, Terms) in the knowledge graph.
- **Interactivity:**
 - **Hover-over:** Metaphactory's built-in functionality will display a pop-up with a natural language description of the node, pulled from its `rdfs:comment` or `skos:note` properties.
 - **Node-Click:** A single click on a related concept node (e.g., "User Retention") will automatically generate a new SPARQL query with that concept as a filter, refreshing the search results and the dynamic filters in the sidebar.

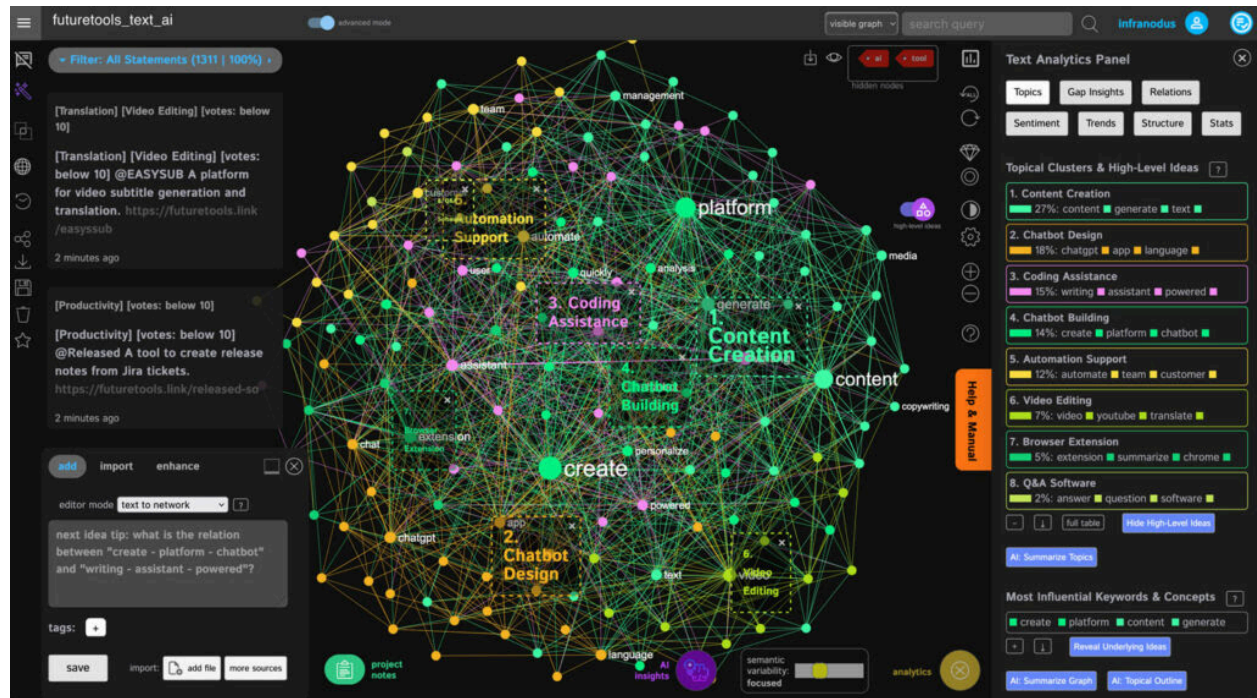
4. The Expert-Curated Searches Library

- **Implementation:** A dedicated Metaphactory dashboard page.
- **Personalization:** The dashboard's primary SPARQL query will be filtered to show only `knowledge:SearchArtifact` instances that are tagged with the current user's role and business domain.
- **Card Anatomy:** The cards for each expert search will be rendered by a custom widget. It will display the search title, a summary from the `rdfs:comment` property, and the name of the expert from a `knowledge:curatedBy` property. The Run This Search button will trigger the execution of the saved SPARQL query.
- **Preserving Searches:** An expert can save a complex search by creating a new `knowledge:SearchArtifact` in the knowledge graph, complete with the SPARQL query, a natural language description, and relevant metadata like `knowledge:curatedBy` and `knowledge:businessDomain`. This is a standard Metaphactory function, ensuring these valuable assets are preserved and shared.

This comprehensive approach leverages Metaphactory's powerful, native knowledge graph

capabilities to build a UI that not only looks great but also provides a deeply intuitive and semantically rich experience for every user in the organization.

Below is a working example of a working example showcasing some of the ideas above



Future State Design

UI/UX Design: Enterprise Semantic Search Interface

The design for this semantic search moves beyond a traditional single-page application and into a truly immersive, proactive, and multi-modal interface. This is a system that doesn't just respond to commands—it anticipates them, understands intent, and synthesizes complex data into a seamless, actionable reality. It's less of a tool and more of a cognitive partner. The aesthetic is clean, immersive, and data-rich, designed for speed, precision, and an almost effortless user experience.

Core Design Philosophy: Contextual Intelligence

The system is built on a foundation of **situational awareness**. It actively monitors the user's role, current projects, and physical environment to proactively surface the most relevant information without a query. Interaction is primarily through natural language and gesture, with tactile controls and visualizations available for deep-dive analysis. The interface is a translucent, dynamic overlay on the user's field of view, presenting a unified "cognitive map" of the company's knowledge.

Snapshot 1: The "Situational Awareness" Interface

Look and Feel: A calm, semi-transparent heads-up display (HUD) that floats in the user's peripheral vision. There is no central search bar. Instead, the system's ambient intelligence presents a few key, high-relevance items.

- **Primary Element:** A central, holographic "Thought Bubble" where the user can initiate a query with a gesture or a natural language command ("Veronica, find..."). This is the conversational entry point.
- **Personalized Proactivity:** In the lower-right corner, a dynamic "Knowledge Stream" displays three items most relevant to the user's current context.
 - **Example for a user in the "Sales" department:**
 - A small card icon appears with a brief text: "Project 'Phoenix' Q3 Sales Data has been updated. John Doe's model shows a 5% increase in North America."
 - A second card: "Your colleague Jane Smith just published her 'Top 5 Customer Churn Factors' analysis."
- **System Feedback:** Subtle, natural language cues appear as the user focuses on an item. If they glance at the Project card, the system might whisper, "Shall I load the interactive dashboard for Project Phoenix?"

Snapshot 2: The "Data Synthesis" Panel

Look and Feel: The search command ("Find the North America Q3 sales report") doesn't bring up a new page. Instead, a knowledge graph in an adaptable panel is presented, displaying a synthesized view of the data. The results are shown in clusters of colored, categorized objects.

- **Main Results Panel:** A central cluster of data assets is arranged in 2D space.
 - **Visual Hierarchy:** Reports, data sets, and dashboards are different shapes and colors (e.g., reports are glowing cylinders, dashboards are flat planes). Their size and proximity to the center indicate their relevance.
 - **Dynamic Filtering:** The filters can be presented in a sidebar. The user can type a command ("Filter for my team's assets") or use the on screen sliders to apply a filter. The system responds with a natural language confirmation and a visual rearrangement of the assets.
- **Contextual Sliders:** Instead of a traditional slider, the user can manipulate a 2D axis or "relevance slider."
 - **Interaction:** The user grabs a glowing horizontal bar labeled "Persona Relevance" and drags it. As they drag, the system provides real-time, natural language feedback: "Increasing focus on 'Director of Sales' insights..." and the assets visually re-sort themselves.
- **Result Card Interaction:** Focusing on an individual asset causes it to expand, revealing a brief, natural language summary of its content and a "trust score" based on the underlying data quality metadata.

Snapshot 3: The "Cognitive Map" Explorer

Look and Feel: This is the ultimate immersive experience. A command like "Explore relationships in the North American market" transports the user into a full, interactive knowledge graph visualizer. The user can navigate this in an intuitive manner.

- **Visualization:** Nodes are no longer flat boxes but are glowing, pulsing spheres. Edges are luminous, animated lines showing the flow of relationships.
- **Interaction:**
 - **Flight Controls:** The user can navigate the map using NLP commands ("Zoom in on Project Phoenix," "Show me the connections to Jane Smith").
 - **Information Overlays:** As the user gets closer to a node (e.g., "Customer Churn"), its semantic definition, key metrics, and related expert searches appear in a highlighted overlay.
- **Expert Insights:** A command like "Show me a subject matter expert's path" highlights a glowing trail through the graph, tracing the connections John Doe made while creating his analysis. This isn't just about data; it's about understanding how a human expert thought.

Snapshot 4: The "Thought Library" - Preserving Expert Journeys

Look and Feel: A clean, minimalist library of saved "thought models." This is the ultimate "expert search" feature, where a complex analysis is saved not as a static report, but as a repeatable cognitive journey.

- **Content:** The library displays virtual "trophies" or artifacts, each representing a complex search.
 - **Example:**
 - **2D Object:** A small, rotating model of a data cluster.
 - **Title:** "John Doe's Q1 Churn Diagnostics."
 - **Metadata:** "Saved on March 20th," "Tags: Churn, Predictive Model, Sales."
- **Interaction:**
 - **Activation:** The user selects the object, and a voice (Veronica) explains: "This model was built by John Doe to identify early indicators of customer churn. It uses data from the CRM, sales, and support logs. Would you like to run it on the current data?"
 - **Run & Learn:** Running the model recreates the cognitive map, showing the user the data sources, filters, and logical steps that the expert took to arrive at their conclusion. This isn't just about getting a result; it's about learning the expert's thought process.

Snapshot 5: The "Diagnostics" Overlay - Expert Control

Look and Feel: A command like "Show me the diagnostics" or a specific gesture opens an expert-only overlay. The beautifully simple UI melts away to reveal the raw, powerful data beneath.

- **Main View:** The translucent background becomes a grid of scrolling code and data streams.
- **Key Elements:**
 - **Real-time SPARQL:** A panel displays the live SPARQL queries being executed by the UI, allowing the expert to debug and fine-tune the semantic logic.
 - **Performance Metrics:** Another panel shows real-time system performance data: query latency, database load, and data quality scores for different sources.
 - **Ontology Viewer:** A pop-out window displays the underlying ontology schema, allowing the expert to modify a concept's definition or a relationship's cardinality on the fly.
- **Control:** All of this is interactable. The expert can use gestures or voice to "pause the query," "inspect the payload," or "update the ontology." This is the ultimate tool for a leader who needs to get under the hood and diagnose a problem at its source, maintaining absolute control over the system.